Universidad de San Carlos de Guatemala

Facultad de Ingeniería
PRACTICAS INICIALES
Sección: C
Inga. Floriza Avila

Informe 4

Desarrollo Web

Nombre: Kristel Amarilis Méndez Marroquin

Carné: 202403155

Fecha:19/03/2026

Descripción General

El presente proyecto consiste en el desarrollo de un backend utilizando Node.js, conectado a una base de datos MySQL.

El sistema permite consultar información de cursos y catedráticos mediante una API REST, la cual puede ser consumida utilizando herramientas como Postman.

Tecnologías Utilizadas

- Node.js
- Express
- MySQL
- MySQL Workbench
- Postman

Base de Datos

Nombre:

practica_web

Tablas creadas:

- Usuario
- Curso
- Catedratico
- Publicacion
- Comentario
- Curso_Aprobado

Características:

- Uso de claves primarias (PRIMARY KEY)
- Uso de claves foráneas (FOREIGN KEY)
- Datos iniciales en tablas de catálogo (Curso y Catedratico)

Configuración del Backend

Instalación de dependencias:

npm install express mysql2 cors bcryptjs jsonwebtoken

Ejecución del servidor:

node server.js

Servidor disponible en:

http://localhost:3001

Conexión a la Base de Datos

Se utiliza el paquete mysql2 para la conexión:

```
const db = mysql.createConnection({
    host: "localhost",
    user: "root",
    password: "********",
    database: "practica_web"
});
```

Endpoints Implementados

Registro de usuario

Método: POST

URL: http://localhost:3001/api/auth/registro

Body (JSON):

```
{
  "registro": "202300100",
  "nombres": "Juan",
  "apellidos": "Perez",
  "correo": "juan@gmail.com",
  "contrasena": "1234"
}
```

Lógica:

- Se reciben los datos del usuario
- La contraseña se encripta usando bcrypt
- Se guarda en la base de datos

Login

Método: POST

URL: http://localhost:3001/api/auth/login

Body:

```
{
  "registro": "202300100",
  "contrasena": "1234"
}
```

Endpoint Base

- Método: GET
- URL:
  http://localhost:3001/

Descripción:

Verifica que el servidor esté funcionando correctamente.

Respuesta:

Backend funcionando correctamente

Obtener Cursos

- Método: GET
- URL:
  http://localhost:3001/api/cursos

Lógica implementada:

- Se realiza una consulta SQL a la tabla Curso
- Se obtienen todos los registros
- Se retornan en formato JSON

Código:

```
app.get("/api/cursos", (req, res) => {
    const sql = "SELECT * FROM Curso";


    db.query(sql, (err, results) => {
        if (err) {
            return res.status(500).json(err);
        }
        res.json(results);
    });
});
```

Obtener Catedráticos

- Método: GET
- URL: http://localhost:3001/api/catedraticos

Lógica implementada:

- Se consulta la tabla Catedratico
- Se obtienen todos los registros

Pruebas con Postman

Se realizaron pruebas de los endpoints mediante Postman.

Endpoints probados:

1. GET /
2. GET /api/cursos
3. GET /api/catedraticos

Evidencias:

http://localhost:3001/api/auth/registro

Save

POST http://localhost:3001/api/auth/registro Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
  "registro": "202300100",
  "nombres": "Juan",
  "apellidos": "Perez",
  "correo": "juan@gmail.com",
  "contrasena": "1234"
}
```

Body Cookies Headers (8) Test Results Status: 200 OK Time: 127 ms Size: 299 B Save Response

Pretty Raw Preview Visualize JSON

```
{
  "mensaje": "Usuario registrado"
}
```

http://localhost:3001/api/auth/login

Save

POST http://localhost:3001/api/auth/login Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary JSON Beautify

```
{
  "registro": "202300100",
  "contrasena": "1234"
}
```

Body Cookies Headers (8) Test Results Status: 200 OK Time: 159 ms Size: 417 B Save Response

Pretty Raw Preview Visualize JSON

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiaWF0IjoxNzczODk4OTcxLCJleHAiOjE3NzM5MDI1NzF9.MGflrUARPTUhyRD8Jtsb2PUMiNfY4aueESYQ50m9gWc"
}
```

http://localhost:3001/api/cursos

GET http://localhost:3001/api/catedraticos

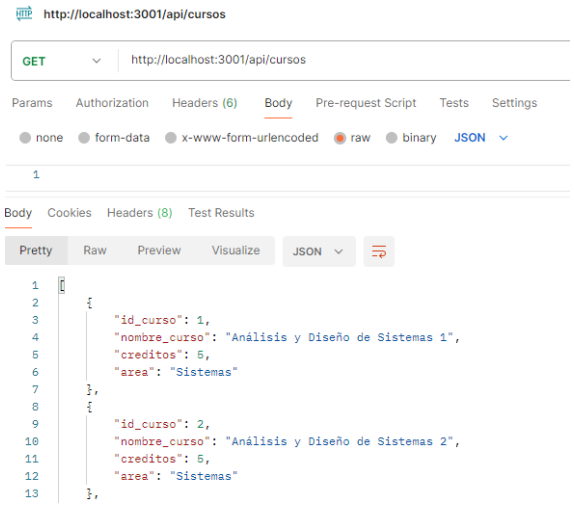
Params Authorization Headers (6) Body Pre-request Script Tests Settings

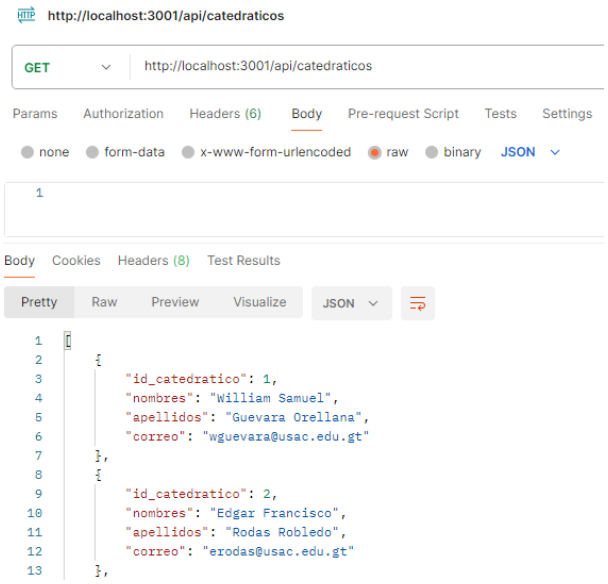
none form-data x-www-form-urlencoded raw binary JSON

Body Cookies Headers (8) Test Results

Pretty Raw Preview Visualize JSON

```
[
    {
        "id_catedratico": 1,
        "nombres": "William Samuel",
        "apellidos": "Guevara Orellana",
        "correo": "wguevara@usac.edu.gt"
    },
```





Consultas SQL Implementadas

Se implementaron consultas para cumplir con los requisitos del proyecto:

Consulta 1:

SELECT nombre_curso, creditos FROM Curso;

Consulta 2:

SELECT nombres, apellidos FROM Catedratico;

Consulta 3 (JOIN):

SELECT u.nombres, c.nombre_curso

FROM Usuario u

JOIN Curso_Aprobado ca ON u.id_usuario = ca.id_usuario

JOIN Curso c ON ca.id_curso = c.id_curso;

Result Grid | Filter Rows:

nombres	apellidos
William Samuel	Guevara Orellana
Edgar Francisco	Rodas Robledo
Claudia Liceth	Rojas Morales
Mirna Ivonne	Aldana Larrazabal
Otto Rene	Escobar Leiva
Gabriel Alejandro	Díaz López
Luis Fernando	Espino Barrios
Luis Alberto	Arias
Edgar Rene	Ornelis Hoil
Sergio Arnaldo	Méndez Aguilar

database* | catedratico | curso

Limit to 1000 rows

```sql
('Luis Alberto', 'Arias', 'larias@usac.edu.gt'),
('Edgar Rene', 'Ornelis Hoil', 'eornelis@usac.edu.gt'),
('Sergio Arnaldo', 'Méndez Aguilar', 'smendez@usac.edu.gt');

SELECT * FROM Curso;

-- Consulta 1: listar cursos y sus créditos
SELECT nombre_curso, creditos
FROM Curso;

-- Consulta 2: listar nombres completos de catedráticos
SELECT nombres, apellidos
FROM Catedratico;

-- Consulta 3: cursos aprobados por usuarios (con JOIN)
SELECT u.nombres, c.nombre_curso
FROM Usuario u
JOIN Curso_Aprobado ca ON u.id_usuario = ca.id_usuario
JOIN Curso c ON ca.id_curso = c.id_curso;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre_curso	creditos
Análisis y Diseño de Sistemas 1	5
Análisis y Diseño de Sistemas 2	5
Arquitectura de Computadoras y Ensambladores 1	4
Arquitectura de Computadoras y Ensambladores 2	4
Bases de Datos 1	5
Bases de Datos 2	5

Curso 4 | Curso 5 | Catedratico 6 | Result 7

Lógica General del Sistema

- El servidor recibe peticiones HTTP
- Se procesan mediante rutas (endpoints)
- Se ejecutan consultas SQL en MySQL
- Se retornan resultados en formato JSON

Conclusión

Se logró implementar un backend funcional con Node.js y MySQL, estableciendo una conexión exitosa entre el servidor y la base de datos.

Se desarrollaron endpoints REST que permiten consultar información de manera estructurada, cumpliendo con los requisitos establecidos en la práctica.